

Relatório Técnico — CardiolIA (Fase 7: Horizontes Inteligentes)

Disciplina: FIAP — Fase 7 (Horizontes Inteligentes)

Grupo: 30

Projeto: Plataforma CardiolIA — Sistema Preditivo Cardiológico com IA e IoT

1. Visão Geral da Solução

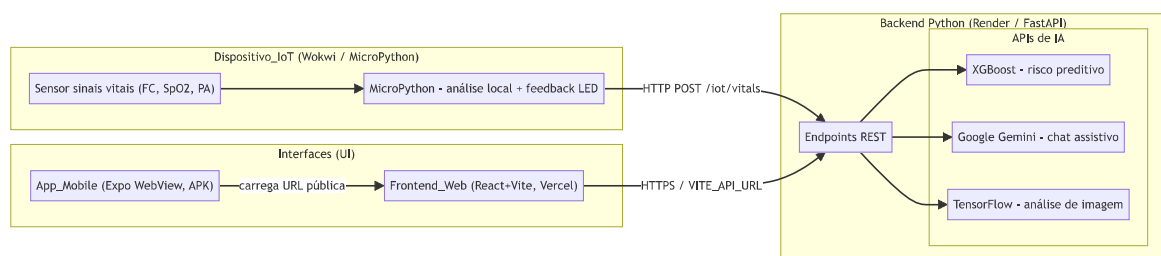
A **Plataforma CardiolIA** é um MVP de triagem cardiológica que integra, de ponta a ponta, um dispositivo IoT simulado, um backend de IA e interfaces web e mobile. A solução unifica os módulos das fases anteriores em uma entrega profissional publicada em ambientes de nuvem (Vercel e Render), com aplicativo Android distribuído via APK (Expo/EAS Build).

A plataforma é composta por quatro camadas:

- **Dispositivo_IoT** — simulação MicroPython no Wokwi que lê sinais vitais, executa análise local e envia os dados ao backend.
- **Backend_CardiolIA** — serviço FastAPI (hospedado no Render) que expõe os endpoints REST e orquestra os três motores de IA.
- **APIs de IA** — XGBoost (risco preditivo), LLM Google Gemini (chat assistivo) e TensorFlow (análise de imagem).
- **Interfaces (UI)** — Frontend_Web (React + Vite, na Vercel) e App_Mobile (Expo WebView).

2. Diagrama de Arquitetura

O diagrama representa a sequência exigida: **Sensor** → **MicroPython** → **Backend Python** → **APIs de IA** → **UI**, identificando os motores de IA e as interfaces de usuário.



Fluxo: Sensor → MicroPython (análise local + LED) → HTTP POST /iot/vitals → Backend FastAPI → XGBoost (risco) + Gemini (chat) + TensorFlow (imagem) → Frontend_Web (Vercel)

+ *App_Mobile (APK)*

3. Fluxo de Dados (IoT → Backend → IA → UI)

- Sensor → MicroPython.** O Dispositivo_IoT no Wokwi realiza a leitura simulada de sinais vitais (frequência cardíaca, SpO2 e pressão arterial sistólica).
- Análise local + feedback visual.** O MicroPython classifica cada leitura como normal ou alterada (ex.: spo2 < 92 ou heart_rate > 110) e aciona um feedback visual (LED) quando a leitura está fora da faixa.
- MicroPython → Backend.** Ao final de cada ciclo, o dispositivo envia os sinais vitais via HTTP POST /iot/vitals para a URL pública do Backend_CardioIA.
- Backend → APIs de IA.** O backend FastAPI orquestra os motores:
 - GET /risk/{id} → XGBoost retorna a predição de risco (alto, moderado, baixo).
 - POST /chat → LLM Google Gemini retorna a resposta do assistente conversacional.
 - POST /images/analyze → TensorFlow retorna o rótulo e a probabilidade da análise.
- Backend → UI.** O Frontend_Web consome os endpoints via VITE_API_URL sobre HTTPS. Caso o backend esteja indisponível, o frontend degrada exibindo dados mock de fallback.
- UI Mobile.** O App_Mobile (Expo WebView) carrega a URL pública do Frontend_Web na Vercel.

Endpoints principais

Método	Rota	Motor / Resultado
GET	/health	Status do serviço
GET	/dashboard/summary	Resumo de pacientes
GET	/patients, /patients/{id}	Dados de pacientes
GET	/vitals/{id}	Sinais vitais
GET	/risk/{id}	XGBoost — risco preditivo
GET	/monitoring/summary	Resumo de monitoramento
POST	/chat	LLM Google Gemini — chat assistivo
POST	/iot/vitals	Ingestão dos dados IoT
POST	/images/analyze	TensorFlow — análise de imagem

4. Stack Tecnológica

Camada	Tecnologia	Hospedagem
Dispositivo_IoT	MicroPython	Wokwi (simulação)
Backend_CardiolIA	Python + FastAPI	Render (Docker)
IA preditiva	XGBoost / scikit-learn	Backend_CardiolIA
IA conversacional	LLM Google Gemini	API Google Gemini (via backend)
IA de imagem	TensorFlow / Keras	Backend_CardiolIA
Frontend_Web	React 19 + Vite	Vercel
App_Mobile	Expo + react-native-webview	APK via EAS Build

A **segurança da chave de API** é garantida mantendo a credencial exclusivamente no backend (variável de ambiente no Render), nunca no bundle do frontend, com os arquivos .env fora do versionamento.

5. Integrantes do Grupo 30

Nome	Função
Ana Beatriz Duarte Domingues	Desenvolvimento / Integração
Junior Rodrigues da Silva	Desenvolvimento / Integração
Carlos Emilio Castillo Estrada	Desenvolvimento / Integração

Professores — Tutor: Lucas Gomes Moreira · Coordenador: André Godoi Chiovato